Keecas Quarto Example

Table of contents

How to use keecas in a Quarto document

This example demonstrates all the key features of `keecas` for symbolic and units-aware calculations in a Quarto document.

> **Note**
>
> If the rendering engine is `KaTeX` (i.e. Jupyter notebook), the label command `\label{}` will result in a `ParseError`. The latex code will work when rendering by `quarto render`, but since you may want to see the result before rendering the document, an option to disable the `\label{}` command is provided.
> When rendering with `quarto` you can have two config:
>
> - if `qmd` files, then everything is fine

- if `ipynb` files, then pass the `--execute` flag to `quarto render` to rerender the notebook

Basic Symbolic Math with Units

Define symbols and calculate basic engineering quantities:

$$F = 10\,\mathrm{kN} \qquad \text{applied force} \tag{1}$$

$$A = 50\,\mathrm{cm}^2 \qquad \text{cross-sectional area} \tag{2}$$

$$\sigma = \frac{F}{A} = 2.00\,\mathrm{MPa} \qquad \text{normal stress} \tag{3}$$

Pipe Commands Demonstration

Using pipe operators for functional composition:

$$x = 3\,\mathrm{m} \tag{4}$$

$$y = 4\,\mathrm{m} \tag{5}$$

$$d = x^2 + y^2 = 25.0\,\mathrm{m}^2 \tag{6}$$

Different LaTeX Environments

Cases Environment

Equation Environment with Labels

Calculate beam deflection with cross-references:

$$\delta = \frac{5\,q\,L^4}{384\,E\,I} = 15.95\,\mathrm{mm} \tag{7}$$

> **Note**
>
> labels emitted by `show_eqn` are latex labels, therefore they need to be referenced with `\eqref{}` or `\ref{}` latex commands.
> **?@eq-QUARTO_EXAMPLE-delta** will not work

The deflection calculated in **??** shows acceptable values for serviceability.

Verification Function

Using the `check` function for design checks:

$$\sigma_{Sd} = 20\,\text{MPa} \tag{8}$$

$$\tau_{Sd} = 15\,\text{MPa} \tag{9}$$

$$\sigma_{Rd} = 250\,\text{MPa} \tag{10}$$

$$\tau_{Rd} = 75\,\text{MPa} \tag{11}$$

$$\frac{\sigma_{Sd}}{\sigma_{Rd}} = 0.080 \quad [\leq 1.0 \quad \textbf{VERIFIED}] \tag{12}$$

$$\frac{\tau_{Sd}}{\tau_{Rd}} = 0.200 \quad [\leq 1.0 \quad \textbf{VERIFIED}] \tag{13}$$

The type of comparison in the `check` function can be specified, as well the value to be specified against.

$$a = 3 \quad [> 1 \quad \textbf{NOT VERIFIED}] \tag{14}$$

$$b = 4 \quad [> 2 \quad \textbf{VERIFIED}] \tag{15}$$

$$c = 5 \quad [\geq 3 \quad \textbf{NOT VERIFIED}] \tag{16}$$

$$d = 6 \quad [\geq 4 \quad \textbf{VERIFIED}] \tag{17}$$

$$f = 7 \quad [\neq 5 \quad \textbf{NOT VERIFIED}] \tag{18}$$

Subs get ordered

Using `pc.subs` will automatically order the substitutions pairs topologically, so you don't have to order them yourself (default `sympy` behavior). For example let's define some mappings in any order:

$$\begin{cases} \delta = \dfrac{5\,q\,L^4}{384\,E\,I} \end{cases} \tag{19}$$

If we use the default behavior of sympy's `subs` method (unsorted substitutions) we get:

$$\begin{cases} x = (1+a)^2 & \text{partially evaluated} \\ y = 4 \\ a = 3 \end{cases} \tag{20}$$

While the default behavior of `pc.subs` is to sort the substitutions:

$$\begin{cases} x = 16 & \text{fully evaluated} \\ y = 4 \\ a = 3 \end{cases} \tag{21}$$

> **Note**
>
> `pc.subs` also take care of sympyfing the object before subsituting. In the above example a is mapped to `3`, an `int`, therefore during dict comprehension the `int` is piped to `pc.subs`. If you would have used the `sympy` subs method directly, you would have get an error because the type `int` does not have `subs` method (the corrected dict comprehension you should look like `S(rhs).subs(...)`).

Check Function Templates

The `check` function supports customizable templates for different visual styles:

$$\text{Default} : [\leq 1.0 \quad \textbf{VERIFIED}] \tag{22}$$

$$\text{Boxed} : \boxed{\leq 1.0 \checkmark \textbf{VERIFIED}} \tag{23}$$

$$\text{Minimal} : \leq 1.0 \checkmark \tag{24}$$

$$a = 3 \tag{25}$$

$$b = 4 \tag{26}$$

$$d = \frac{\sqrt{a^2 + b^2}}{c} = 0.417 \tag{27}$$

$$c = a\,b = 12.000 \tag{28}$$

Custom Templates

You can also use completely custom templates:

$$\text{Pass} : \quad \leq 1.0 \ \textbf{PASS} \tag{29}$$

$$\text{Fail} : \quad > 1.0 \ \textbf{FAIL} \tag{30}$$

Summary

This example demonstrated:

- Basic symbolic math with units
- Pipe command usage (see Section)
- Different LaTeX environments (align, cases, equation)
- Label and cross-reference functionality
- Dataframe operations
- Verification functions
- Complex symbolic manipulations

All calculations in Section show that keecas provides a powerful interface for engineering calculations in Quarto documents.

Template Configuration

Templates can also be configured globally via TOML config files:

```
[check_templates.template_sets.minimal]
success = '${symbol}{rhs} \,\textcolor{{green}}{{\checkmark}}$'
failure = '${symbol}{rhs} \,\textcolor{{red}}{{\times}}$'
```

This allows you to set project-wide or user-wide styling for all check functions.

Test using a `QuartoMarkdownBase64`

````
```{=latex}
\begin{align}
a  &  = 5{\,}\text{m}  &     \\[8pt]
b  &  = 12{\,}\text{m}  &  \quad\text{length b}  \\[8pt]
c  &  = a + b  &  \quad\text{total length c}
\end{align}
```
````

“‘=latex

$$a = 5\,\mathrm{m} \tag{31}$$

$$b = 12\,\mathrm{m} \qquad \text{length b} \tag{32}$$

$$c = a + b \qquad \text{total length c} \tag{33}$$

“‘